

INF221 - [Modern] Web Design & Development

Isaac S. Mwakabira

Learning Outcomes

By the end this lecture, students will be able to:

- ❖ decompose monolithic applications into small, specific modules
 - recognize appropriate service boundaries
 - architect stateful and stateless services to optimize scalability and reliability
 - how does constraints determine API architectural styles
 - build loosely coupled services by implementing a well-designed REST architecture
 - design consistent, standard RESTful service APIs
- ❖ understand the development of web application architecture leading to a more modular approach
- ❖ sketch out the components that would be used in a range of approaches to web application architecture
- ❖ outline a range of benefits and potential problems associated with a specific approach to web application architecture
- ❖ contrast developments in architecture (SOA, Cloud) with more traditional tiered approaches

An introduction to web applications architecture

- ❖ An introduction to web applications architecture
 - provides an overview of the design and implementation of computer software
 - running on web servers, instead of running solely on desktop computers, laptops or mobile devices
- ❖ Web applications are **software** accessible via a **network** using a **web browser**.
 - Websites such as for Chancellor College, forums and online banking systems are some examples of web applications.
- ❖ **Web services** describe a standardised way of integrating web applications using the **internet standards and protocols**.

Architecture

Definition:

- ❖ The term is associated with design of buildings; **structure and style** thereof.
- ❖ In the context of architecting web applications:
 - Structure
 - Physical components -> “**system architecture**”
 - Software - programs making up an application -> “**software architecture**”
- ❖ What is software architecture?
 - “...computational components and interactions among those components” ~ Shaw and Garlan (1996)
 - “...defining an application in terms of its computational components & specifying how these components are related & behavioral interactions among themselves to achieve overall system goals”
 - it also relates to the decisions that are taken about the design and composition of the overall system, how it is broken down into components and how these components cooperate
- ❖ What is system architecture?
 - the configuration of hardware, network and software components of a computer system which satisfies the requirement for the provision a particular application or service.
 - A ‘system architecture’ holds the key to success or failure of a system every bit as much as the ‘software architecture’ does for the software.

Software Behavior vs Architecture

Every software must have behavior & architectural values.

It is your responsibility to ensure both values are satisfied and in a balanced manner.

- ❖ **Behavior** - software must satisfy user requirements. Your job is to test that functional requirements are met. **Testing** & debugging.
- ❖ **Architecture** - software must be “softly” **easy to understand and change**. It must have a shape that **can easily be adapted to new requirements**.

What about **Function** vs **Architecture**?

- ❖ Balance is key, **software must be soft**.
 - If you give me a program that works perfectly but is impossible to change, then it won't work when the requirements change, and I won't be able to make it work.
 - therefore the program will become useless over time
 - If you give me a program that does not work but is easy to change, then I can make it work, and keep it working as requirements change.
 - therefore the program will remain continually useful during its lifetime

Web Application Architecture

Every engineer must be able to answer the following questions:

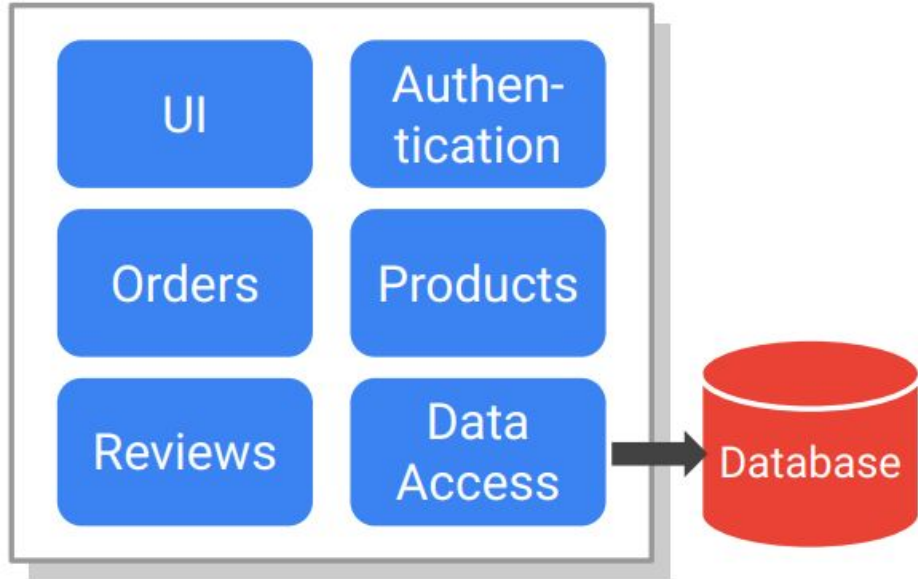
- ❖ How can we scale business systems?
- ❖ How many requests do we need to handle in our application?
- ❖ How many seconds latency is acceptable for our architecture?

Businesses grow with time, how can we evolve applications built around such entities to remain relevant in the market? Web applications evolution must therefore be based on:

- ❖ **Availability**
 - number of downtimes, is the application always up or down?
- ❖ **Scalability**
 - a measure of how well the application is served to end-users. Should be able to serve millions of users without noticeable downtimes.
- ❖ **Reliability**
 - does every user get to do their job via the application without any problems
- ❖ **Efficiency**
 - should respond acceptable latency like < 2 sec — short response latency

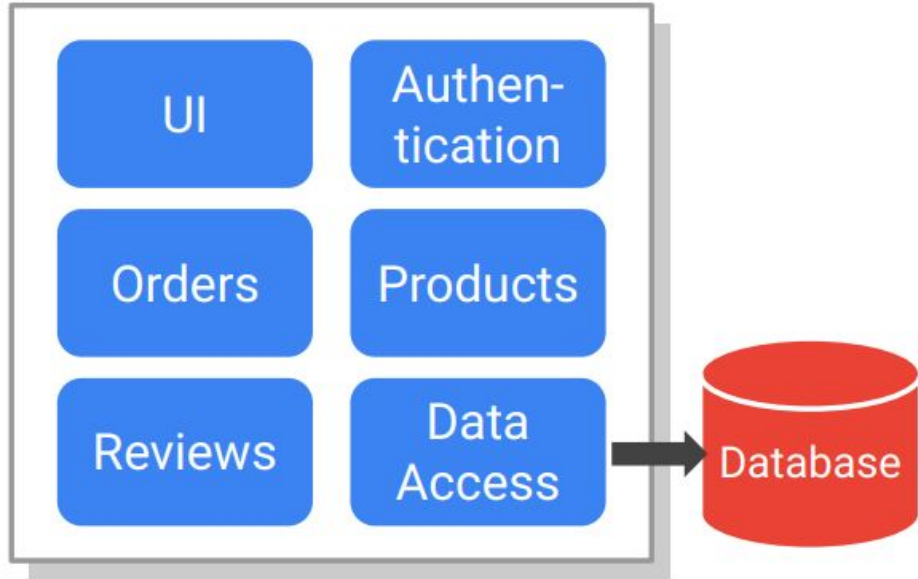
Traditional Web Applications: Monolithic

- ❖ Traditionally, web applications have been monolithic in nature
- ❖ Implement all features in a single code base with a database for all data
- ❖ All components are packaged at deployment time and deployed together on a single web server
 - includes user interface, business codes and database calls in single code base
 - application concerns are contained in a single big deployment



Traditional Web Applications: Monolithic

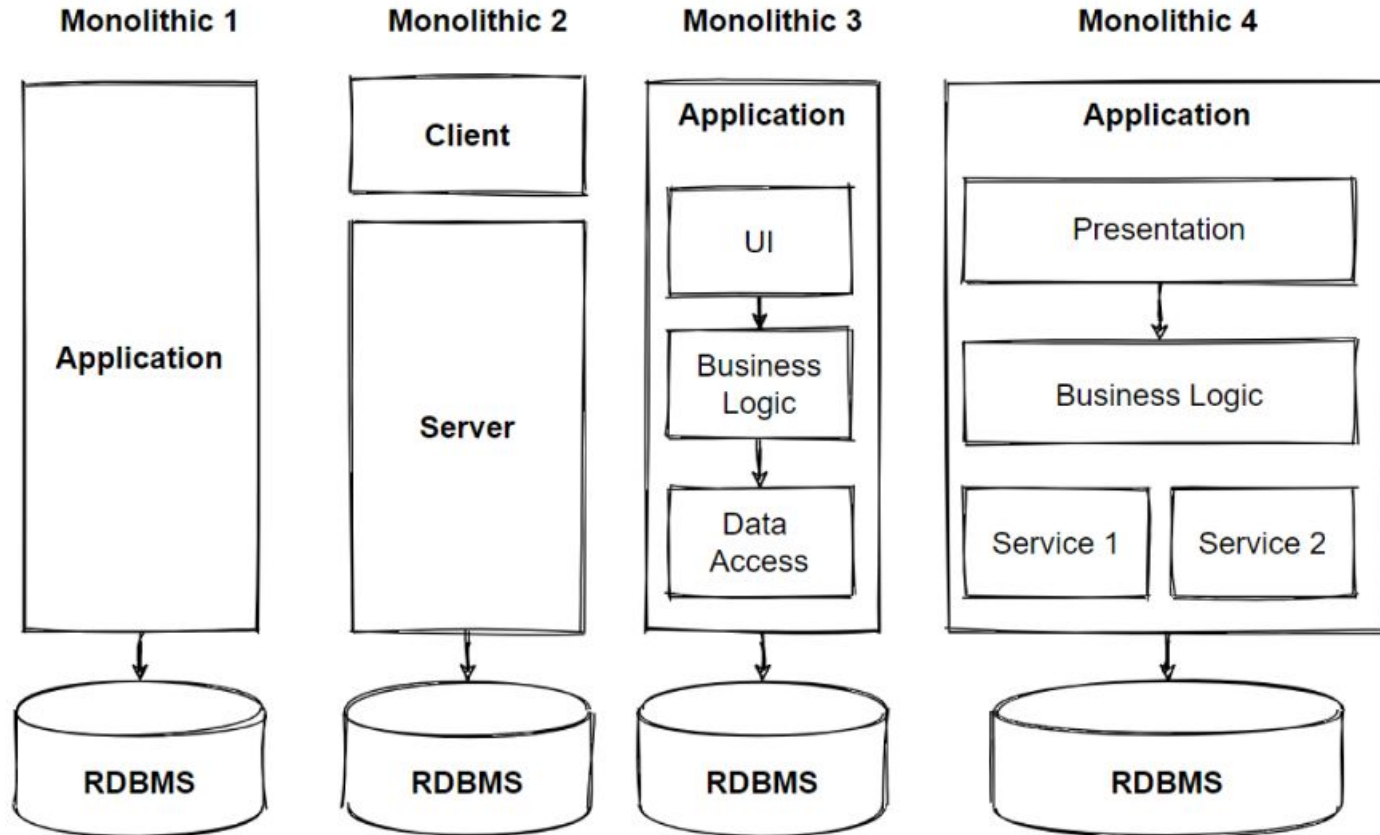
- ❖ This approach has many disadvantages, is inefficient and ineffective for several reasons:
 - location of developers
 - difficult to work in parallel on same code base
 - integration with legacy systems
 - hard to implement new features on legacy big monolithic applications
 - business constraints & resource management
 - some departments may be priorities & others not
 - difficult to scale systems, & modify architectures, difficult to manage with increase in code base
 - any change, requires deploying a new version of the entire application



Traditional Web Applications: When to Use Monolithic

- ❖ Are we building a small application? What is the size of our application? For starters this is the best architecture we can use, for now, if size is small.
 - easy to build
 - test
 - deploy
 - maintain
 - troubleshoot when errors occur during runtime
 - easy and fast vertical scalability (scale up)
- ❖ Activity 1
 - Design a Monolithic Architecture based application using the ideas to be submitted.

Traditional Web Applications: Monolithic vs Evolution



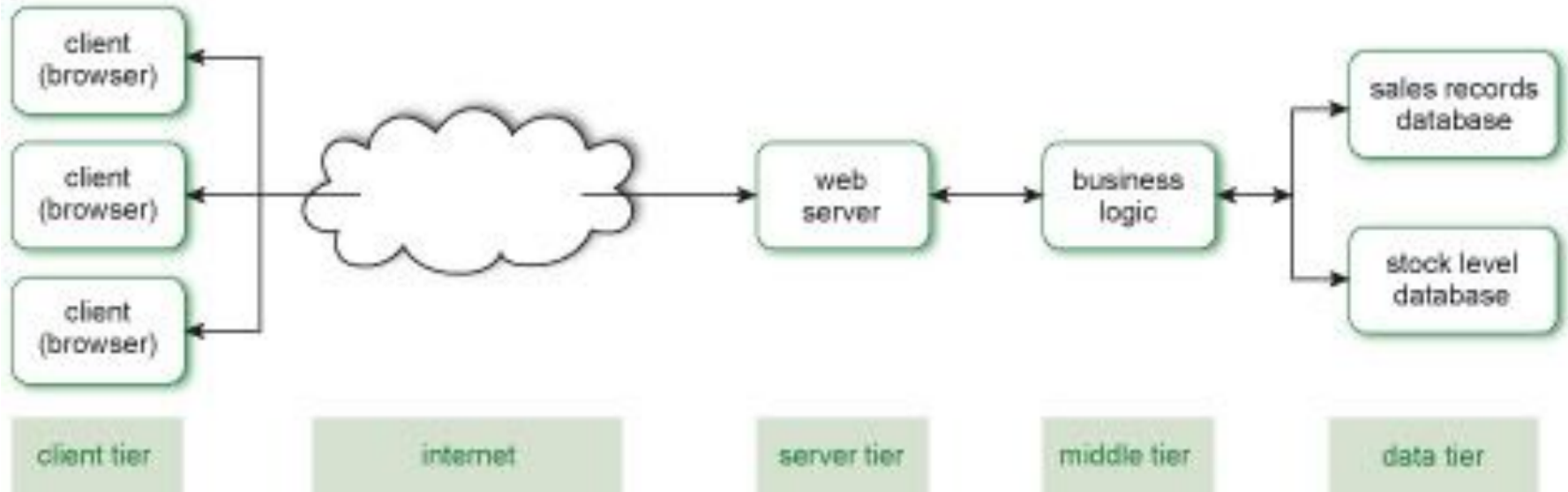
Client–server architecture aka “two-tier architecture”

- ❖ divides an application into two parts, ‘client’ and ‘server’
 - implemented on a computer network, which connects the client to the server.
- ❖ The server provides the central functionality:
 - i.e., any number of clients can connect to the server and request that it performs a task.
- ❖ The server accepts these requests, performs the required task and returns any results to the client, as appropriate, in various forms.



Multi-tier architecture

- ❖ The approach of splitting an application into tiers can be taken further
 - Both the client and the server parts can be further subdivided if this is appropriate for the application



Network and distributed architectures

- ❖ As our description moves from monolithic applications to client–server and then to N-tier, the application has been broken down into more and more parts.
- ❖ This trend has been extended in a modern approach called service-oriented architecture (SOA).
- ❖ SOA breaks down an application into a set of much smaller tasks that can be performed by
 - small independent, ‘software components’, each performing a discrete task commonly called a service

Service Oriented Architectures

- ❖ Solution for integrating diverse systems by providing an architectural style that promotes loose coupling and reuse.
- ❖ The software components provide services to other components via a communications protocol, typically over a network.
- ❖ A service is some functionality, typically a business process, which is packaged as a reusable software component that is:
 - well-defined – a software component with a clearly specified interface and outcome
 - self-contained – the implementation of the service is complete and independent of any product, vendor or technology
 - a black-box – the implementation of the service is hidden (encapsulated) from the service consumer.

Service Oriented Architectures...

- ❖ Solution for integrating diverse systems by providing an architectural style that promotes loose coupling and reuse.
- ❖ The software components provide services to other components via a communications protocol, typically over a network.
- ❖ A service is some functionality, typically a business process, which is packaged as a reusable software component that is:
 - well-defined – a software component with a clearly specified interface and outcome
 - self-contained – the implementation of the service is complete and independent of any product, vendor or technology
 - a black-box – the implementation of the service is hidden (encapsulated) from the service consumer.
- ❖ Principles
 - Interoperability, location transparency, discoverability, loose-coupling & encapsulation, abstraction, autonomy,
 - statelessness, standard interfaces & contracts, reusability, dynamic configuration
- ❖ What are the advantages of developing business applications based on SOA?

Web services

- ❖ When a service is made available over the internet, it is then usually termed a web service
 - describes the service that a client can access from a server over the internet, utilising web protocols and standards to enable the exchange of data between them.
- ❖ The main protocols and standards employed are:
 - XML (eXtensible Markup Language)
 - SOAP (Simple Object Access Protocol)
 - REST (Representational State Transfer)
 - JSON (JavaScript Object Notation) based.

What about microservices?

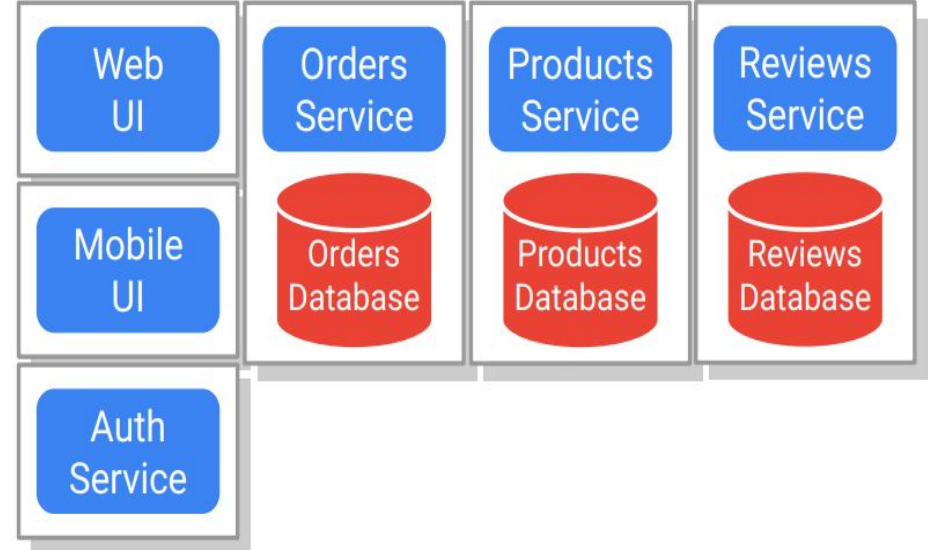
*“The microservice architectural style is an approach to developing a single application as a **suite of small services**, each **running in its own process and communicating with lightweight mechanisms**, often an HTTP or gRPC API” ~*

Martin Fowlers

“loosely coupled and independently deployable smaller components”

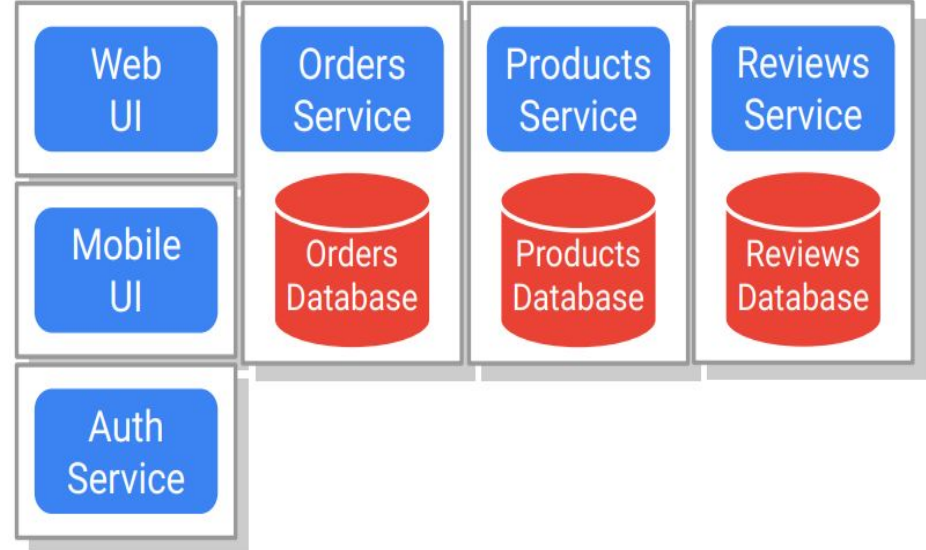
Microservices - Characteristics

- ❖ Small business services that can work together; small, independent, and loosely coupled
 - each service manages its own data
 - small, independent, and loosely coupled.
 - a single small team of developers can write and maintain a service.
 - each service is a separate codebase, which can be managed by a small development team
- ❖ and can be deployed autonomously/independently
 - A team can update an existing service without rebuilding and redeploying the entire application
- ❖ Each service has an independent datastore
 - have their own technology stack, included the database and data management model
 - differs from the traditional model, where a separate data layer handles data persistence
- ❖ Communicate to each other over a combination of REST APIs, event streaming, and message brokers
- ❖ Organized by **business capability**, with the line separating services often referred to as a **bounded context**



What about microservices?

- ❖ Current industry trend
- ❖ Advantages:
 - enable teams to work independently and deliver through to production at their own cadence
 - supports scaling the organization: adding more teams increases speed
 - able to scale the microservices independently based on their requirements
 - Unlike monolith apps, individual components in microservices are deployable



Microservices - Benefits

❖ Agility

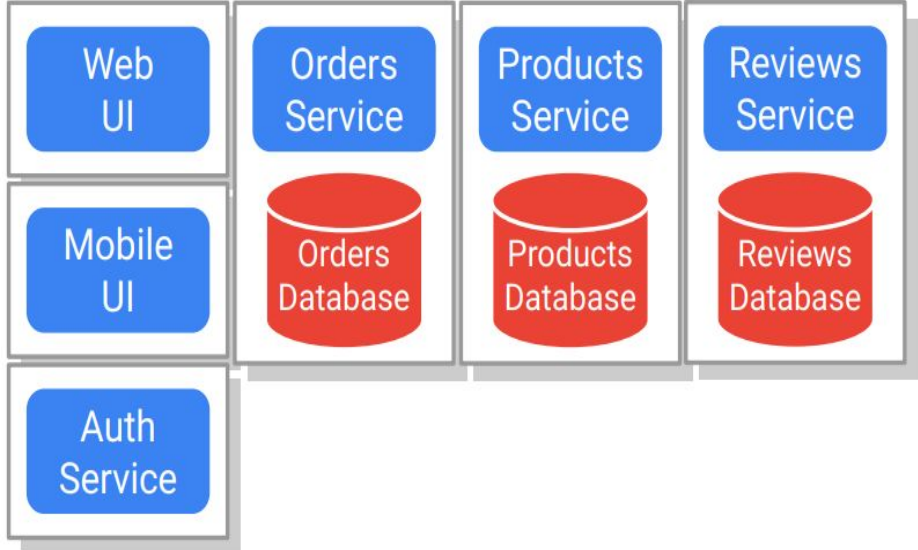
- services are smaller and independently deployable

❖ Small, focused teams

- a microservice should be small enough that a single feature team can build, test, and deploy it

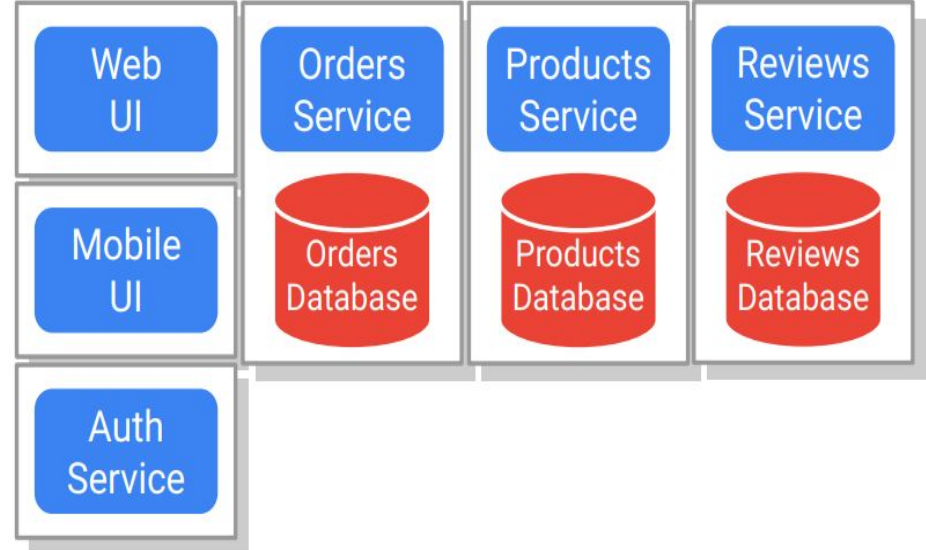
❖ Scalability

- can be scaled independently
 - scale out sub-services that require less resources, without scaling out the entire application



Microservices - Challenges

- ❖ Complexity
 - so many moving parts that are hard to manage
 - microservices application has lots of services that need to work together and should create a value
 - since there are lots of services, that means there is more moving parts than the monolithic application
- ❖ Network problems and latency
 - since microservice are small and communicate with inter-service communication
 - we should manage network problems should one service develop issues
- ❖ Data integrity
 - every microservice has its own data persistence.
 - data consistency can be a challenge



Pros and cons of microservice architectures

Pros	Cons
Easier to develop and maintain	Increased complexity when communicating between services
Reduced risk when deploying new versions	Increased latency across service boundaries
Services scale independently to optimize use of infrastructure	Concerns about securing inter-service traffic
Faster to innovate and add new features	Multiple deployments
Can use different languages and frameworks for different services	Need to ensure that you don't break clients as versions change
Choose the runtime appropriate to each service	Must maintain backward compatibility with clients as the microservice evolve over time

Cloud architectures

SOA is well placed to take advantage of a recent development known as 'cloud technology' based on what is often termed 'the cloud'

Cloud technology has changed how organisations use technology to provide computing services

Whereas SOA provides **services on a network**, the cloud extends the principle to other resources such as processing power and storage facilities and is not limited to the provision of services

Cloud architectures...

Cloud computing is a model for:

- ❖ enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g. networks, servers, storage, applications, and services)
- ❖ that can be rapidly provisioned and released with minimal management effort or service provider interaction

Cloud computing is the practice of:

- ❖ delivering computing services – servers, storage, databases, networking, software, analytics and more – on-demand over the Internet.
- ❖ It is a means of providing computing services as a utility to consumers in the same way as other utilities such as gas and electricity.
- ❖ Companies offering these computing services typically charge for cloud computing services based on usage

Cloud architectures: Essential characteristics

Characteristics:

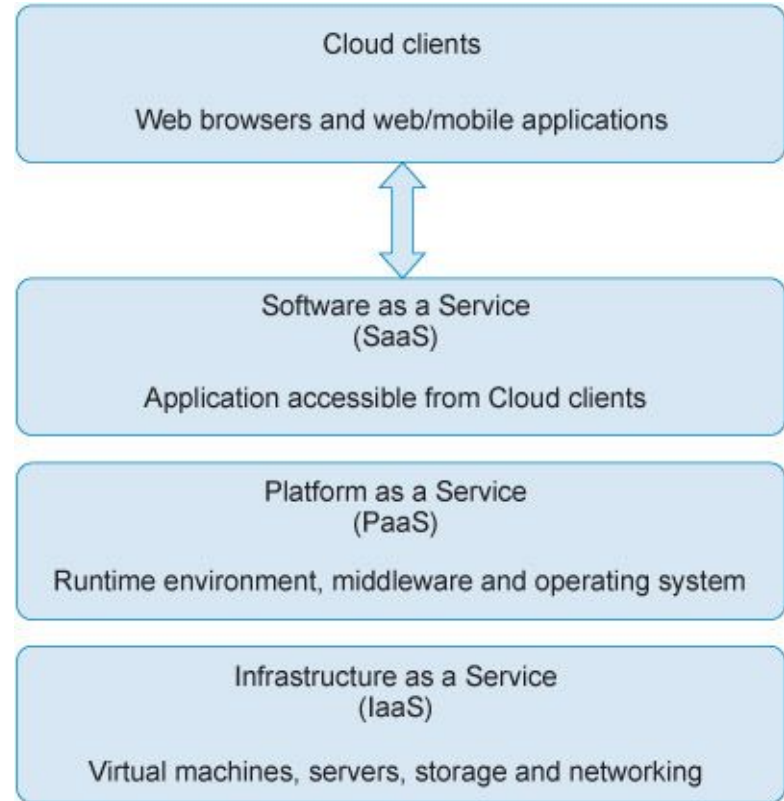
- ❖ On-demand self-service
- ❖ Broad network access
- ❖ Resource pooling
- ❖ Rapid elasticity
- ❖ Measured service

Cloud architectures: Types

Types

- ❖ **Software as a Service (SaaS)**
 - provides applications designed for end-users, delivered over the web
- ❖ **Platform as a Service (PaaS)**
 - provides tools and services for developing and deploying applications
- ❖ **Infrastructure as a Service (IaaS)**
 - provides servers, storage, network services and virtual machines

Examples: **AWS, Microsoft Azure, Google Cloud**



RESTful Web Service APIs

Summary

- ❖ In summary, a ‘software architecture’ relates to
 - the arrangement of software components and the interactions between these components.
 - the decisions that are taken about the design and composition of the overall system, how it is broken down into components and how these components interact.
 - This type of structuring reflects high-level decisions that are difficult to change later because they determine software-level decisions
 - It is thus difficult to change the architecture of a system without starting the design from the beginning
- ❖ Web Applications Architecture concerns
 - the **design and implementation** of **computer software** that runs on **web servers**,
 - instead of running solely on desktop computers, laptops or mobile devices.
- ❖ You explored ‘client–server’ and ‘multi-tier’ approaches to web applications architecture
 - representing mainstream common ‘**tried and tested**’ approaches, and
 - ‘**network type**’ approaches - Service-Oriented Architecture (SOA) and cloud architecture, which represent more recent developments

References

- ❖ Erl, T. (2007), SOA: Principles of Service Design, New Jersey, Prentice Hall.
- ❖ Fowler, M. (2002) Patterns of Enterprise Application Architecture, Boston, Addison-Wesley Longman Publishing.
- ❖ Shaw, M. and Garlan, D. (1996) Software Architecture: Perspectives on an Emerging Discipline, New Jersey, Prentice Hall.
- ❖ <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>
- ❖ <https://leanpub.com/software-architecture-for-developers>
- ❖ <https://nordicapis.com/rest-vs-graphql-how-constraints-determine-api-style/>
- ❖ Pautasso C. (2014) RESTful Web Services: Principles, Patterns, Emerging Technologies. In: Bouguettaya A., Sheng Q., Daniel F. (eds) Web Services Foundations. Springer, New York, NY. https://doi.org/10.1007/978-1-4614-7518-7_2
- ❖ <https://dl.acm.org/doi/10.1145/514183.514185>
- ❖ <https://martinfowler.com/articles/richardsonMaturityModel.html>